

Svelte - Shared Runes

HINWEIS: Diese Unterlagen basieren auf den Skripten von Prof. Grüneis.

Bisher haben wir Runen nur innerhalb einer Komponente bzw. in einer "Master-Detail-Beziehung" verwendet (Komponente beinhaltet andere Komponente und Rune wird via Two-Way-Binding übergeben). Es gibt aber auch die Möglichkeit, Runen über Komponenten hinweg zu teilen, ohne dass diese in einer Master-Detail-Beziehung stehen müssen. Wir bezeichnen das als "Shared Runes".

Die Idee ist, dass wir eine recht lose Kopplung zwischen Komponenten haben, die voneinander gar nichts wissen müssen. Sie teilen sich nur einen **gemeinsamen Zustand** (die Rune) und reagieren auf Änderungen dieses Zustands. Das ist besonders nützlich, wenn wir mehrere Komponenten haben, die auf denselben Daten arbeiten, aber nicht direkt miteinander verbunden sind.

Vor Svelte 5 wurde das mit sogenannten "Stores" realisiert. Diese sind inzwischen aber veraltet, da Svelte 5 eine neue Möglichkeit bietet, Shared Runes zu verwenden, die einfacher und direkter ist.

Beispiel: Gemeinsame Zähler-Rune (noch falsch)

Als Beispiel möchten wir einen gemeinsamen Zähler implementieren. Dies ist einfach eine Zahl, die von mehreren Komponenten gelesen und geändert werden kann. Wenn sich diese Zahl ändert, sollen alle Komponenten, die sie verwenden, automatisch informiert werden.

Am besten erstellt man solche "Shared Runes" in einem eigenen Bereich, z.B. **src/lib/shared**. Dort können wir eine Datei namens **Counter.svelte.js** anlegen, die unseren gemeinsamen Zähler definiert.

```
export let counter = $state(0);
```

WICHTIG: Beachte die Erweiterung `.svelte.js`. Das ist eine Konvention, die Svelte verwendet, um anzuzeigen, dass es sich um eine Datei handelt, die eine Rune definiert.

Um den Zugriff auf diese "Shared Runes" zu vereinfachen, können wir für den Pfad eine Alias-Definition in der `svelte.config.js` hinzufügen:

```
import adapter from '@sveltejs/adapter-auto';

/** @type {import('@sveltejs/kit').Config} */
const config = {
  kit: {
    adapter: adapter(),
    alias: {
      $shared: 'src/lib/shared'
    }
  }
};

export default config;
```

Diese Rune können wir nun in verschiedenen Komponenten verwenden. Zum Beispiel könnten wir eine Komponente namens `src/lib/components/CounterDisplay.svelte` erstellen, die den aktuellen Wert des Zählers anzeigt:

```
<script>
  import { counter } from '$shared/Counter.svelte.js';
</script>
<p>Der aktuelle Zählerstand ist: {counter}</p>
```

Diese Komponente können wir nun auf unserer Startseite (`+page.svelte`) einbinden:

```
<script>
  import CounterDisplay from '$lib/components/CounterDisplay.svelte';
```

```
</script>
```

```
<CounterDisplay />
```

So weit, so gut ... Was aber, wenn wir den Zähler auch verändern möchten? Dafür könnten wir eine weitere Komponente namens **CounterButton.svelte** erstellen, die einen Button enthält, um den Zähler zu erhöhen:

```
<script>
  import { counter } from '$shared/Counter.svelte.js';
</script>
<button onclick={() => counter++}>Erhöhen</button>
<button onclick={() => counter--}>Verringern</button>
```

Leider bekommen wir hier aber eine **Fehlermeldung**, da die Rune `counter` nicht direkt verändert werden kann. Der Grund ist, dass in JavaScript das ganz einfach nicht erlaubt ist (hat also nichts mit Svelte zu tun).

Imported values can only be modified by the exporter

The identifier being imported is a *live binding*, because the module exporting it may re-assign it and the imported value would change. However, the module importing it cannot re-assign it. Still, any module holding an exported object can mutate the object, and the mutated value can be observed by all other modules importing the same value.

JS



```
// my-module.js
export let myValue = 1;
setTimeout(() => {
  myValue = 2;
}, 500);
```

JS



```
// main.js
import { myValue } from "/modules/my-module.js";
import * as myModule from "/modules/my-module.js";

console.log(myValue); // 1
console.log(myModule.myValue); // 1
setTimeout(() => {
  console.log(myValue); // 2; my-module has updated its value
  console.log(myModule.myValue); // 2
  myValue = 3; // TypeError: Assignment to constant variable.
  // The importing module can only read the value but can't re-assign
  it.
}, 1000);
```

Zähler-Rune (fast richtig)

Um das Problem zu lösen, könnten wir die Rune über ein Objekt zur Verfügung stellen:

```
const _counter = $state(0);
```

```
export const counter = { value: _counter };
```

Dann könnten wir über die Property `value` auf den Zähler zugreifen und ihn auch verändern:

```
<script>
  import { counter } from '$shared/Counter.svelte.js';
</script>
<button onclick={() => counter.value++}>Erhöhen</button>
<button onclick={() => counter.value--}>Verringern</button>
```

Jetzt haben wir aber ein neues Problem: Der Wert in der Seite ist immer 0! Das liegt daran, dass im Objekt nicht die Rune als solche, sondern nur jener Wert der Rune auf `value` zugewiesen wird, der beim Erzeugen des Objekts vorliegt.

Der Svelte-Compiler macht uns übrigens auch darüber aufmerksam:

```
src/lib/shared/Counter.svelte.js:3:32 This reference only captures the in
https://svelte.dev/e/state_referenced_locally
```

Zähler-Rune (richtig)

Man muss also innerhalb der Rune ein Objekt zur Verfügung stellen, das man dann verändert:

```
export const counter = $state({ value: 0 });
```

Die Rune `counter` kapselt jetzt also ein Objekt, und weil es eine Rune ist, werden bei jeder Änderung des Objekts alle Abhängigkeiten informiert.



Finale Version

Counter.svelte.js

```
let state = $state({ value: 0 });

export const counter = {
  get value() { return state.value; },
  set value(v: number) { state.value = v; }
};
```

Diese können wir nun wie folgt verwenden:

```
<script>
  import { counter } from '$shared/Counter.svelte.js';
</script>

<p>Der aktuelle Zählerstand ist: {counter.value}</p>
<button onclick={() => counter.value++}>Erhöhen</button>
<button onclick={() => counter.value--}>Verringern</button>
```

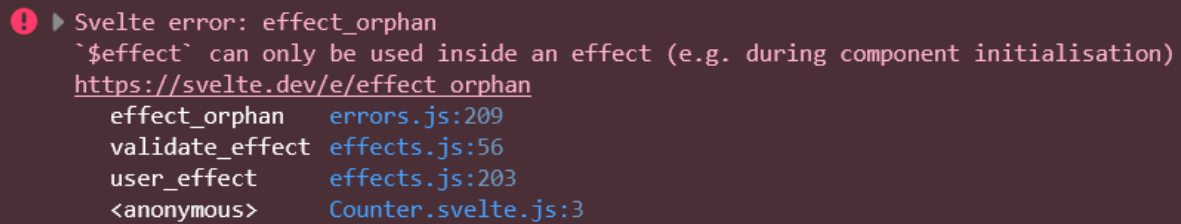
Mit \$effect() auf Änderungen reagieren

Nun möchten wir mithilfe eines "Effekts" Änderungen am Zähler auf der Konsole protokollieren. Wir schreiben also folgendes in die Counter.svelte.js:

```
export const counter = $state({ value: 0 });

$effect(() => {
  console.log(`Der Zähler wurde geändert: ${counter.value}`);
});
```

Nun bekommen wir aber eine andere Fehlermeldung:



```
! ▶ Svelte error: effect_orphan
`$effect` can only be used inside an effect (e.g. during component initialisation)
https://svelte.dev/e/effect\_orphan
effect_orphan      errors.js:209
validate_effect    effects.js:56
user_effect        effects.js:203
<anonymous>       Counter.svelte.js:3
```

Der Grund ist, dass die Rune `counter` in diesem Fall nicht direkt in der Datei definiert ist, sondern über den Export zur Verfügung gestellt wird. Das bedeutet, dass sie nicht direkt im Skript verwendet werden kann, da sie erst nach der Definition verfügbar ist.

In diesem Fall müsste man den Code in ein **`$effect.root()`** verlagern:

```
export const counter = $state({ value: 0 });

$effect.root(() => {
  $effect(() => {
    console.log(`Der Zähler wurde geändert: ${counter.value}`);
  });
});
```

`$effect.root()` ist notwendig, weil **`$effect()`** nur innerhalb eines aktiven **Tracking-Scopes** ausgeführt werden kann, und dieser Scope in einer separaten `.svelte.js`-Datei fehlt.

Tracking-Scopes in Svelte 5

Svelte-Komponenten erzeugen automatisch einen Tracking-Scope während ihrer Initialisierung, in dem **`$effect()`**-Aufrufe reagieren können. In Modul-Dateien wie Ihrer "shared rune"-Datei existiert dieser Scope jedoch nicht – **`$effect()`** würde einen "effect_orphan"-Fehler werfen.

Zweck von **`$effect.root()`**

`$effect.root()` erstellt manuell einen Tracking-Scope außerhalb der Komponenten-Lifecycle. Es erlaubt verschachtelte **`$effect()`**-Aufrufe in beliebigen Kontexten (z. B. Klas-

sen, Module, dynamische Objekte) und gibt ein Cleanup-Funktion zurück, das manuell aufgerufen werden muss.

Alternative: Komponenten-Context

Für geteilte Runes ist es oft besser, `$effect.root()` zu vermeiden und Effects in der Root-Komponente zu platzieren, wo der Scope automatisch verfügbar ist:

```
<script>
  import { counter } from '$lib/Counter.svelte.js';
</script>

{$effect(() => console.log(`Zähler: ${counter.value}`))}
```

Das ist performanter und nutzt Sveltes Lifecycle-Management.

Notifier mit Shared Runes implementieren

Als weiteres Beispiel möchten wir einen "Notifier" für String-Messages implementieren. Komponenten können Nachrichten an diesen Notifier senden, und andere Komponenten können diese Nachrichten anzeigen.

Man braucht also:

- Intern eine Rune zum Speichern des Wertes
- Eine öffentliche Methode zum Schreiben einer neuen Nachricht
- Einen getter zum lesenden Zugriff

Notifier.svelte.js

Erstellen wir unter `$shared` eine Datei namens **Notifier.svelte.js** mit folgendem Inhalt:


```
function createNotifier() {
  let msg = $state('');
  $effect.root(() => $inspect(msg));
  return {
    get message() { return msg; },
    notify(message) {
      if (message.trim().length === 0) return;
      msg = message;
    }
  }
}
```

```
export const notifier = createNotifier();
```

Zum Generieren von Nachrichten erstellen wir eine Komponente **NotifierProducts.svelte**. Diese wird nur die `notify()`-Methode benutzen und muss gar nicht wissen, dass dahinter eine Rune steckt.

```
<script>
  import { notifier } from '$shared/Notifier.svelte';

  let newMessage = $state('Hallo');
</script>

<div>
  <input bind:value={newMessage} />
  <button onclick={() => notifier.notify(newMessage)}>Notify</button>
</div>
```

Die Anzeige der Nachricht erfolgt in einer weiteren Komponente **NotifierListener.svelte**. Sie muss nur den Getter `message` verwenden, um die aktuelle Nachricht zu lesen:

```
<script>
```

```
import { notifier } from '$shared/Notifier.svelte';
import { untrack } from 'svelte';

let messages = $state([]);

$effect(() => {
  notifier.message; // required as trigger
  untrack(() => messages.push(notifier.message));
});
</script>
```

Last message: {notifier.message}

```
<div>
  <ul>
    {#each messages as message, index (index)}
      {#if message.length}
        <li>{message}</li>
      {/if}
    {/each}
  </ul>
</div>
```

`untrack()` verhindert, dass `notifier.message` innerhalb des `$effect` unnötig als reaktive Dependency getrackt wird.

Reaktivität von `$effect`

`$effect` trackt automatisch alle reaktiven Werte (Runes, Signals), die innerhalb seines Callbacks gelesen werden, und führt sich bei deren Änderung erneut aus. Ohne `untrack` würde `notifier.message` sowohl als Trigger (`notifier.message;`) als auch im `push`-Aufruf getrackt – doppelt unnötig.

Zweck von `untrack()`

`untrack(fn)` führt `fn` ohne Tracking-Kontext aus: Reactive Werte innerhalb werden nicht als Dependencies registriert. Der Code liest `notifier.message` zweimal:

Erstes Mal: `notifier.message`; -> soll tracken (Trigger)

Zweites Mal: `messages.push(notifier.message)` -> soll nicht tracken

Ohne `untrack` (fehlerhaft):

```
$effect(() => {  
  notifier.message; // trackt  
  messages.push(notifier.message); // trackt erneut  
});
```

Mit `untrack` (korrekt):

```
$effect(() => {  
  notifier.message; // trackt nur hier  
  untrack(() => messages.push(notifier.message)); // trackt nicht  
});
```